



Hyperbridge Protocol

Report
name:

**handleConsensus accepts
attacker-supplied IHost, permanently
bricking epoch attribution**

PREPARED BY

HackenProof

REPORT ID

HYPERBR-44

AUTHOR

Petrus

REPORT CREATED

14.05.2026 13:57:56 +0000

SEVERITY

Critical

VULNERABILITY

Business Logic Errors

TARGET

<https://github.com/polytope-labs/hyperbridge>

VULNERABILITY DETAILS

Function: `handleConsensus` in `evm/src/core/HandlerV2.sol`

Affected code (full, as in the repo)

```
modifier notFrozen(IHost host) {
    FrozenStatus state = host.frozen();
    if (state == FrozenStatus.Incoming || state == FrozenStatus.All) revert HostFrozen;
    _;
}
```

```
function handleConsensus(IHost host, bytes calldata proof) external notFrozen(host) {
    uint256 delay = block.timestamp - host.consensusUpdateTime();
    if (delay >= host.unStakingPeriod()) revert ConsensusClientExpired();

    bytes memory previousState = host.consensusState();
    (bytes memory verifiedState, IntermediateState[] memory intermediates, uint256 latestHeight)
        = ICensususV2(host.consensusClient()).verify(previousState, proof);

    if (keccak256(previousState) == keccak256(verifiedState)) return;
    host.storeConsensusState(verifiedState);

    uint256 intermediatesLen = intermediates.length;
    for (uint256 i = 0; i < intermediatesLen; i++) {
        IntermediateState memory intermediate = intermediates[i];
        uint256 latestHeight = host.latestStateMachineHeight(intermediate.stateMachineId);
        if (latestHeight != 0 && intermediate.height > latestHeight) {
            StateMachineHeight memory stateMachineHeight =
                StateMachineHeight({stateMachineId: intermediate.stateMachineId, height: intermediate.height});
            host.storeStateMachineCommitment(stateMachineHeight, intermediate.commitment);
        }
    }

    if (nextAuthoritySetId > _currentEpoch) {
        _currentEpoch = nextAuthoritySetId;
        _epochs[nextAuthoritySetId] = msg.sender;
        emit NewEpoch(nextAuthoritySetId, msg.sender);
    }
}
```

```
}  
}  
}
```

Why it's a bug (precise + simple)

handleConsensus takes an IHost host argument from the caller but never checks that host is a real, registered host. Every read inside the function (host.frozen(), host.consensusClient(), host.consensusState(), etc.) is delegated to whatever contract the caller passed in. An attacker can deploy a fake IHost and a fake IConsensusV2 whose verify(...) returns nextAuthoritySetId = type(uint256).max. The handler then writes to its own storage:

```
_currentEpoch = type(uint256).max;  
_epochs[type(uint256).max] = msg.sender;
```

Because _currentEpoch is now the maximum uint256, the guard nextAuthoritySetId > _currentEpoch can never be true again, and no legitimate epoch will ever be recorded by handleConsensus for the lifetime of this HandlerV2 contract.

Function where the bug lives

HandlerV2.handleConsensus (evm/src/core/HandlerV2.sol). The contract's own _currentEpoch and _epochs storage is corrupted; the real host is untouched.

Impact (one sentence)

On-chain relay attribution (relayOf(epochId) / currentEpoch()) is permanently poisoned, so any reward/accounting system that reads these views to credit relayers will silently stop recording new epochs.

Reachable in production (one sentence)

handleConsensus is external with no caller restriction beyond the notFrozen check (which is evaluated on the supplied host), so anyone with gas on a chain hosting a deployed HandlerV2 can call it once with a malicious host argument.

Attacker benefit (one sentence)

An attacker can grief relayers and the protocol's on-chain accounting at near-zero cost (one transaction), and could trivially front-run the first legitimate handleConsensus to install themselves as _epochs[max] for whatever vanity that yields.

Effect on users (one sentence)

End users of the bridge are not directly harmed because real message passing still works (the real host's consensus state continues to update on legitimate calls), but relayers reliant on on-chain epoch attribution may lose payouts or face accounting drift that downstream affects fee economics.

Best fix (one sentence)

Replace the arbitrary IHost host parameter with the handler's immutable, canonical host (or maintain a whitelist), so only the real host can ever drive epoch state.

// Option A — immutable canonical host:

IHost public immutable HOST;

function handleConsensus(bytes calldata proof) external {

if (HOST.frozen() == FrozenStatus.Incoming || HOST.frozen() == FrozenStatus.All) revert
HostFrozen();

// ... use HOST in place of `host` everywhere ...

}

// Option B — whitelist registered hosts:

mapping(address => bool) private _registeredHosts;

function handleConsensus(IHost host, bytes calldata proof) external notFrozen(host) {

if (!_registeredHosts[address(host)]) revert UnknownHost();

// ... existing body ...

}

VALIDATION STEPS

Attack path

Attacker deploys EvilHost and EvilConsensusClient.

EvilHost.frozen() returns FrozenStatus.None; EvilHost.consensusUpdateTime() returns block.timestamp; EvilHost.unStakingPeriod() returns a large number; EvilHost.consensusClient() returns EvilConsensusClient; EvilHost.consensusState() returns bytes("a"); EvilHost.storeConsensusState(...) is a no-op.

EvilConsensusClient.verify(prev, proof) returns (bytes("b"), new IntermediateState, type(uint256).max).

Attacker calls HandlerV2.handleConsensus(EvilHost, "").

The handler reaches `_currentEpoch = type(uint256).max`; `_epochs[max] = attacker`; and emits `NewEpoch(max, attacker)`.

Every future call with the real host has `nextAuthoritySetId < type(uint256).max`, so the if block never executes again — `_epochs[realId]` is never written.

Reference

evm/src/core/HandlerV2.sol — `notFrozen(IHost host)` modifier, `handleConsensus`, `_currentEpoch`, `_epochs`, `relayerOf`, `currentEpoch`.

evm/src/core/EvmHost.sol — `latestStateMachineHeight`, `storeStateMachineCommitment` (only invoked on whichever host was supplied — so the real host is left intact, which is why this is attribution corruption rather than full bridge compromise).